Twelve two-base-pair helices: an exact designability bound for a fixed family in the maxStacks model

Tatsuhiko Takimoto

25 October 2026 Version 1.2

Abstract

Fix a family of noncrossing RNA targets with one enclosing two-base-pair helix, $D-1$ inner two-base-pair helices, and one unpaired nucleotide in each hairpin and between consecutive helices around the central loop. In the maxStacks model with GU pairs allowed, no isolated pairs, and minimum pair separation $j-i \geq 2$, the largest designable value of D is 12. A 71-nucleotide sequence makes the $D=12$ target the unique stack maximizer. Nonexistence for $D=13$ follows from a nucleotide-level CNF and one DRAT refutation, also checked after conversion to LRAT by a verified CakeML checker. For every $D \geq 14$, a family-wise embedding reduces a putative design to a 97-position relaxation whose 34 finite cases are inconsistent. We give the normalization and local replacement arguments connecting these certificates to the model, and separate their verification obligations. The one-nucleotide hairpin convention is essential to the stated target family; the result does not concern the usual minimum of three unpaired hairpin nucleotides.

1 The question

Boury, Bulteau and Ponty [1] formulate inverse folding under stack maximization and show how sufficiently long helices permit design at large loop degree. Their Lemma 1 excludes a loop with degree Δ and equal helix length h when $h < \log_6 \Delta$. At $h=2$, this gives the necessary condition $\Delta \leq 36$; it does not assert that degree 36 is attainable. We are grateful for their model and degree–length obstruction, which supply the question studied here.

We determine the maximum for one fixed family with helices of length two and specified unpaired positions. Under the model below, a target with twelve such helices admits a unique optimal fold, whereas every target in this family with thirteen or more does not. The contribution is this exact value for the fixed infinite family. The model, local exchange obstructions and the use of finite certificates are not claimed as new methods.

The proof has three parts: an explicit design at $D=12$, a direct refutation at $D=13$, and a separate argument for all $D \geq 14$. The last part is needed because failure at $D=13$ alone does not imply failure at larger D. Local replacements explain the finite reductions; the certificate for $D=13$ can be checked without trusting the auxiliary graph and context encodings used to construct it.

2 Model, target family and theorem

Let $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{U}\}$ and let the symmetric compatibility relation $\sim$ consist of

$$\mathrm{AU},\ \mathrm{UA},\ \mathrm{CG},\ \mathrm{GC},\ \mathrm{GU},\ \mathrm{UG}.$$

A structure P on positions $0, \ldots, n-1$ is a set of pairs (i,j), $i<j$, with disjoint endpoints. For any two pairs $(i,j),(k,l) \in P$, the crossing $i<k<j<l$ is forbidden. Every pair must satisfy $j-i \geq 2$ and have a neighboring nested pair: at least one of $(i+1, j-1)$ and $(i-1, j+1)$ also

belongs to P. Call a structure satisfying these requirements a *legal structure*; in particular, it has no isolated pairs. A structure is compatible with a sequence w if $w_i \sim w_j$ for every $(i,j) \in P$. Its score is

$$\mathrm{st}(P) = \bigl|\{(i,j) \in P : (i+1, j-1) \in P\}\bigr|.$$

A helix is a maximal run of consecutive nested pairs. A helix of ℓ pairs contributes $\ell - 1$ stacks, so a two-base-pair helix contributes one stack. The pair-separation condition permits a hairpin with just one unpaired nucleotide. It is the theoretical convention of [1, Section 2], not the usual RNA convention requiring at least three.

For $D \geq 2$, put $n_D = 6D - 1$ and $s_k = 6k - 3$ for $1 \leq k \leq D - 1$. Define

$$T_D = \{(0, n_D - 1), (1, n_D - 2)\} \;\cup\; \bigcup_{k=1}^{D-1} \{(s_k, s_k + 4), (s_k + 1, s_k + 3)\}. \tag{1}$$

Equivalently, its dot-bracket string is

$$\texttt{((.}\quad \underbrace{\texttt{((.)).}\;\cdots\;\texttt{((.)).}}_{D-1 \text{ copies}}\quad \texttt{)).}$$

Whitespace in this display is only typographic. The enclosing helix and the $D - 1$ inner helices give score D. Thus D counts all helices, including the enclosing one, and is the degree of the central loop. The unpaired positions between branches will be called separators.

Definition 1. A design of T_D is a sequence compatible with T_D for which every other compatible legal structure $P \neq T_D$ satisfies $\mathrm{st}(P) < D$. Let D_2 be the largest $D \geq 2$ for which such a sequence exists.

Theorem 2. *In the model of Section 2, T_{12} is designable and T_D is not designable for every $D \geq 13$. Consequently $D_2 = 12$.*

3 A design attaining twelve

The following sequence has length 71:

`GAAAGCCUAGGACCACCCGGACGACGAACAGUACAAUGAGUAACAUGACACUAAUAAAUAAUCGCAGCAUC`

Two dynamic programs, one using free-interval and paired-boundary states and one enumerating maximal helices, return optimum 12 and exactly one optimal structure, namely T_{12}. The bundled checker `checkers/core.py` implements the maximal-helix recurrence below; `checkers/check_finite.py` compares it with direct enumeration.

The maximal-helix decomposition explains both optimization and counting. For an interval $[i,j]$, let $M_0(i,j)$ be the best score over all legal structures on that interval, and let $M_1(i,j)$ be the best score when the pair (i,j) is forbidden. Retain the number of optimal structures alongside each score. A structure either leaves i unpaired or starts with a maximal helix

$$H(i,k,\ell) = \{(i+r, k-r) : 0 \leq r < \ell\}, \qquad \ell \geq 2.$$

Require compatibility of all these pairs and $k - i - 2(\ell - 1) \geq 2$. Its score contribution is

$$(\ell - 1) + M_1(i + \ell, k - \ell) + M_0(k + 1, j). \tag{2}$$

For $M_1(i,j)$, exclude $k = j$; the unpaired option uses $M_0(i+1, j)$. Empty intervals have score zero and one structure. The inner state in (2) forbids extension of the chosen helix, so a structure has exactly one decomposition. Counts multiply over the two intervals and add over equal-score alternatives. This counts structures rather than different parses of the same structure.

A separately written implementation of this decomposition also returns $(12, 1, T_{12})$. It was compared with direct enumeration of noncrossing matchings, followed by the legality tests, on 22,485 short sequences. The finite comparison supports the implementation; the unique decomposition is the mathematical reason for the recurrence. No general claim about designability at all smaller D is needed.

4 Normalization and local obstructions

Write $\mathcal{W} = \{\mathrm{AU}, \mathrm{UA}, \mathrm{CG}, \mathrm{GC}\}$ for Watson–Crick pairs, and $\overline{a}$ for the unique Watson–Crick complement of a.

Lemma 3 (Normalization)**.** *If T_D has a design, it has one whose target pairs all lie in $\mathcal{W}$ and whose target-unpaired positions all lie in $\{\mathrm{A}, \mathrm{C}\}$.*

Proof. The compatible-neighbor sets satisfy $N(\mathrm{A}) = \{\mathrm{U}\} \subset N(\mathrm{G}) = \{\mathrm{C}, \mathrm{U}\}$ and $N(\mathrm{C}) = \{\mathrm{G}\} \subset N(\mathrm{U}) = \{\mathrm{A}, \mathrm{G}\}$. Replacing a single G by A, or U by C, cannot create a compatible pair. Apply these replacements at target-unpaired positions. At each target GU or UG pair, replace its G by A, making it a Watson–Crick pair. Each position belongs to at most one target pair, so the changes can be made sequentially while preserving the target. They only remove compatible competitors. The target therefore remains uniquely optimal. □

This is an existence-preserving transformation, not a condition that every original design must already satisfy. The global permutation $\tau : \mathrm{A} \leftrightarrow \mathrm{C}$, $\mathrm{G} \leftrightarrow \mathrm{U}$ preserves compatibility, $\mathcal{W}$ and $\{\mathrm{A}, \mathrm{C}\}$. We use it either to fix one separator or to choose an orbit representative, never to impose both restrictions without an additional argument.

Call a legal structure different from T_D with score at least D a *rival*. An incompatible pair in every rival is necessary for a design. Two replacement operations provide many such obstructions.

Lemma 4 (Swap and three-pair replacement)**.** *Let two inner branches start at $s < t$. Replacing their four pairs by*

$$(s, t+4),\ (s+1, t+3),\ (s+3, t+1),\ (s+4, t) \tag{3}$$

gives a rival. Replacing the enclosing helix and the branch at s by

$$(0, s+1),\ (1, s),\ (s+3, n_D - 1),\ (s+4, n_D - 2) \tag{4}$$

also gives a rival. Alternatively, choose consecutive three-position blocks starting at $x < y$, each within the seven-position window of one of two branches or at an enclosing-helix end. If $y - x \geq 4$, replace the two original helices by

$$K(x, y) = \{(x, y+2), (x+1, y+1), (x+2, y)\}. \tag{5}$$

For the choices specified below, this is a rival as well.

Proof. For (3), $t - s \geq 6$, so the shortest pair has separation $t - s - 4 \geq 2$. The four pairs are nested and form two stacks. Branches between s and t lie inside the innermost new pair; the remaining branches are disjoint from the new pairs or enclose them. In (4), the two new helices are disjoint and enclose the branches before and after s, respectively. Their shortest pairs also have separation at least two.

In (5), the innermost separation is $y - x - 2 \geq 2$. The new helix has three pairs and two stacks, replacing two stacks. A branch window contains only that branch's paired positions and target-unpaired positions. Hence none of the retained target pairs uses a new endpoint. For the inner and enclosing choices below, all retained branches lie inside a new pair or in a disjoint interval. There is no crossing or isolated pair. Each replacement changes the target and preserves at least its score; additional stacks can only increase that score. □

4.1 A graph on 32 words

A normalized inner branch is the five-letter word

$$v = (a, b, h, \overline{b}, \overline{a}), \qquad a, b \in \Sigma, \quad h \in \{\mathrm{A}, \mathrm{C}\}.$$

There are 32 words. Join distinct words when their (a, b) types agree, or when some internal consecutive triplet in one is reverse-compatible with an internal consecutive triplet in the other. Reverse-compatible means that all three opposing letter pairs are compatible. This graph has 66 edges, obtained by evaluating these finite predicates. Repeated types permit (3). For internal triplets, offsets from branch start lie in $\{0, 1, 2\}$, so their start separation is at least $6 - 2 = 4$, permitting (5). Thus the $D - 1$ inner words of any normalized design form an independent set, with no repeated word or type.

4.2 The neighboring separators

Extend a word to its seven-letter context

$$L\ a\ b\ h\ \bar{b}\ \bar{a}\ R, \qquad L, R \in \{\mathrm{A}, \mathrm{C}\}.$$

The consecutive triplets start at offsets $-1, 0, 1, 2, 3$ from the position of a. Two contexts with offsets r, q satisfying $|r - q| \leq 2$ admit a legal three-pair replacement in either branch order. For a single window, the six legal fragments of score at least one other than the original `.((.)).` are

$$\texttt{((.))..}, \quad \texttt{..((.))}, \quad \texttt{((..)).},$$
$$\texttt{.((..))}, \quad \texttt{((...))}, \quad \texttt{(((.)))}.$$

Each replaces that branch while the other target pairs remain.

For the enclosing helix, the end triplets occupy $[0, 2]$ and $[n_D - 3, n_D - 1]$. Its context consists of the four paired letters and the two separators at 2 and $n_D - 3$. Besides the outer swap (4), its relevant replacements are

left triplet:	$K(0, s + r), \quad r \in \{1, 2, 3\},$
right triplet:	$K(s + r, n_D - 3), \quad r \in \{-1, 0, 1\},$
first shift:	$\{(0, n_D - 2), (1, n_D - 3)\},$
second shift:	$\{(1, n_D - 1), (2, n_D - 2)\}.$

Here $s = 6j - 3$ is an actual inner-branch start. The triplet moves remove the enclosing helix and that branch; a shift removes just the enclosing helix. In each case all other target pairs are retained.

The actual contexts share separators between consecutive branches. Allowing their copies to vary independently relaxes the necessary conditions. A solution of this relaxation need not be a design. Its inconsistency, together with the embedding below, excludes a design.

5 Excluding every $D \geq 14$

5.1 A relaxation on 97 positions

Let the auxiliary target have dot-bracket string

$$\texttt{((. }\underbrace{\texttt{.((.)). } \cdots \texttt{ .((.)).}}_{\text{13 copies}}\texttt{ .))}.$$

It has 97 positions, enclosing pairs $(0, 96), (1, 95)$, external separators at $2, 94$, and thirteen seven-position blocks $3 + 7k, \ldots, 9 + 7k$ for $0 \leq k \leq 12$. The auxiliary branch starts are $u_k = 4 + 7k$; a block reads $L\,a\,b\,h\,\bar{b}\,\bar{a}\,R$. All target pairs are Watson–Crick and all target-unpaired letters are in $\{\mathrm{A}, \mathrm{C}\}$, with no separator fixed by color symmetry.

Define $\mathcal{U}_{14}$ to be the CNF requiring these conditions and forbidding the 1,731 rivals in Table 1. All unspecified target pairs are retained. Nucleotide variables are one-hot, and a variable p_{ij} is equivalent to compatibility of the two letters, using the encoding in Section 6.1. Here p_{ij} is introduced for every $i < j$, including adjacent positions. This is just a compatibility variable; every rival still obeys $j - i \geq 2$. The formula has 5,044 variables and $67,129 + 1,731 = 68,860$ clauses.

Rival family	Replacement	Count
One window	Replace its branch by any of the six other legal seven-position fragments with score at least one.	78
Inner swap	Use (3) with $(s,t)=(u,v)$ for two auxiliary branches.	78
Outer swap	Use (4) with $s=u$ and $n_D=97$.	13
Inner triplets	$K(u+r,v+q)$, $u<v$, $r,q\in\{-1,0,1,2,3\}$, $\lvert r-q\rvert\le 2$.	1,482
Left outer triplet	$K(0,u+r)$, $r\in\{1,2,3\}$.	39
Right outer triplet	$K(u+r,94)$, $r\in\{-1,0,1\}$.	39
Outer shift	Replace the enclosing pairs by $\{(0,95),(1,94)\}$ or $\{(1,96),(2,95)\}$.	2

Table 1: The seven families of auxiliary rivals, with auxiliary branch starts $u,v\in\{4+7k : 0\le k\le 12\}$. A triplet replacement removes the two original helices it uses. Each structure is legal, differs from the auxiliary target, and has score at least 14.

5.2 Covering the finite possibilities

The thirteen words have distinct (a,b) types and form an independent set in the 32-vertex graph. There are

$$\binom{16}{13}2^{13}=4,587,520$$

type-and-hairpin choices before the graph restrictions. An exhaustive enumeration leaves 68 sets of thirteen words. Each rejected choice contains a forbidden pair supported by an actual swap or internal triplet rival. The retained sets split into 34 two-element orbits under τ.

To pass from word sets to fixed cases, one must also account for their order. Each of the twelve adjacent permutations of the seven-position blocks, and the letter permutation τ, preserves the multiset of *all clauses* of $\mathcal{U}_{14}$. This was checked by literal substitution. The adjacent permutations generate every block permutation. Therefore any model can be transformed to one with words in lexicographic order (using A < C < G < U) and belonging to one of the 34 representatives. Their thirteen-word sets are listed in `data/U14-cases.json`. This is a symmetry of the selected formula, not an assertion about arbitrary RNA structures.

Each case adds 65 nucleotide units fixing the five letters in each of thirteen branches. Its CNF has 68,925 clauses. All 34 cases are inconsistent. In addition to certificate acceptance by the public DRAT checker, a separate unit-propagation implementation produces a contradiction in every case, and a replay checks the reason clause for each propagation. Thus this finite step can be checked using the case inputs alone, without the supplied DRAT additions.

Proposition 5. *The formula $\mathcal{U}_{14}$ is unsatisfiable.*

Proof. Every model would have one of the 68 surviving word sets. Formula symmetry carries it to one of the 34 fixed cases, contradicting that case's checked inconsistency. □

5.3 Returning a rival to the original target

Lemma 6. *A design of T_D for any $D\ge 14$ would give a model of $\mathcal{U}_{14}$.*

Proof. Normalize the design and choose any thirteen inner branches, indexed $j_0<\cdots<j_{12}$. Copy the letters at actual positions $0,1,n_D-2,n_D-1$ to auxiliary positions $0,1,95,96$ and those at $2,n_D-3$ to $2,94$. For each block set

$$\phi(3+7k+r)=6j_k-4+r,\qquad 0\le r\le 6. \tag{6}$$

Assign each auxiliary letter the letter at its image. The target and unpaired-letter clauses hold, and assign every p_{ij} its actual compatibility truth value.

The map ϕ is not injective on all positions: consecutive selected branches can share a separator, and the first or last block can share a separator with an enclosing end. We check that every listed rival still gives a valid replacement in the original structure.

For a one-window rival, the seven positions stay consecutive and distinct. Its endpoints outside the branch were target-unpaired, so all other target helices may be retained. For an inner triplet rival, actual branch starts $s = 6j_a - 3 < t = 6j_b - 3$ satisfy $t - s \geq 6$. Hence

$$(t + q) - (s + r) \geq 6 - 2 = 4.$$

The two triplets have distinct endpoints and legal separation. The potential shared-separator collision would require the left offset $r = 3$ and right offset $q = -1$, which is excluded by $|r - q| \leq 2$. Retained branches between the chosen ones are enclosed by the new helix; those outside do not cross it.

For a left outer triplet, $s \geq 3$ and $r \geq 1$ give $s + r \geq 4$. For a right outer triplet, $s \leq n_D - 8$ and $r \leq 1$ give $(n_D - 3) - (s + r) \geq 4$. These same restrictions prevent use of a shared separator twice. Retained branches lie inside the new helix or in a disjoint interval.

Inner swaps use only paired positions, with shortest new separation $t - s - 4 \geq 2$. Outer swaps have shortest pairs $(1, s)$ and $(s+4, n_D-2)$, again of separation at least two. Their nesting and disjointness follow the proof of Lemma 4. For an outer shift, the enclosing endpoints and external separator remain consecutive actual positions, so the shifted helix remains legal and encloses every retained inner branch.

In every family, the replaced one or two target stacks are replaced by at least as many stacks. Add all untouched target helices, including unselected branches. The resulting structure is noncrossing, has no isolated pair, differs from T_D, and has score at least D. If the auxiliary rival were compatible, the same letter pairs would make this actual rival compatible, contradicting the design. Thus all rival-exclusion clauses of $\mathcal{U}_{14}$ hold, giving the asserted model. □

Proposition 5 and Lemma 6 exclude all $D \geq 14$. Tests of finitely many embeddings supplement this argument; the inequalities and family-wise construction establish its universal quantifier. This route uses an auxiliary target, not a direct single refutation for the canonical T_{14}.

6 Excluding $D = 13$ by one pure CNF

6.1 Nucleotides, compatibility and rivals

For the 77 positions of T_{13}, introduce one-hot variables $x_{i,a}$ for $a \in \Sigma$ and variables p_{ij} for $j - i \geq 2$. Require Watson–Crick target pairs and A/C at target-unpaired positions. Use τ once to fix the first separator, position 2, to A.

For each compatible ordered letter pair $a \sim b$, add $\neg x_{i,a} \vee \neg x_{j,b} \vee p_{ij}$. For each $a \in \Sigma$, also add

$$\neg p_{ij} \vee \neg x_{i,a} \vee \bigvee_{b \in N(a)} x_{j,b}, \tag{7}$$

and its endpoint-reversed version. The six forward clauses and eight support clauses, with one-hot letters, make p_{ij} equivalent to actual compatibility. Redundant units set target p_{ij} true and target-unpaired/target-unpaired p_{ij} false.

Let $\mathcal{R}_{13}$ be the supplied list of 227,280 distinct rivals. For every $P \in \mathcal{R}_{13}$, add

$$\bigvee_{(i,j) \in P \setminus T_{13}} \neg p_{ij}. \tag{8}$$

Every listed structure has been checked for balanced brackets, disjoint endpoints, noncrossing, minimum separation, absence of isolated pairs, difference from the target, and score at least 13.

There are 227,255 rivals of score 13 and 25 of score 14. The list need not contain every legal structure: any design must avoid every member of any such list.

The resulting formula F has 3,158 variables: 308 nucleotide variables and 2,850 compatibility variables. Its 41,128 base clauses comprise 539 one-hot clauses, 312 target Watson–Crick exclusions, 50 unpaired restrictions, one separator unit, 39,900 compatibility clauses, and 326 redundant units. Adding the rivals gives 268,408 clauses. The input has no word, set or context variables.

Proposition 7. *The formula F is unsatisfiable, and T_{13} has no design.*

Proof. Unsatisfiability is certified by the single checked DRAT stream identified in Appendix A. A hypothetical design could be normalized by Lemma 3, then color-swapped to fix position 2. Assign its letters and their compatibility values to x and p. All base clauses would hold. Since it is a design, each legal rival has an incompatible pair, which cannot be a target pair; hence (8) holds. This would satisfy F, a contradiction. □

6.2 How the refutation is assembled

The graph and context reductions organize the certificate's construction. There are 1,267 independent word sets of size twelve. Context obstructions dispose of 1,123, and position-dependent cases dispose of the remaining 144. In the final certificate, word and context indicators are introduced as fresh definitions within the proof. Case conclusions are guarded by the negation of their conditions, so no case condition becomes an assumption of the input F.

Prefix-occurrence variables record which words occur in the first k branch positions. The proof rules out the final word sets and propagates these exclusions backwards to the empty prefix, ending in the empty clause. Definitions are checked by the resolution asymmetric tautology (RAT) rule; subsequent reverse unit propagation (RUP) steps and deletions are part of the same stream. Its maximum variable index is 8,727, although F itself has only 3,158 variables. The checker verifies the additions against the original input and preceding proof state. Consequently, neither the older auxiliary encodings nor an assertion that the 1,267-set list is complete is an extra premise of Proposition 7.

The design in Section 3, Proposition 7, and the universal exclusion in Section 5 prove Theorem 2.

7 Computer assistance and proof obligations

Table 2 separates the arguments that must be read mathematically from the finite objects checked by programs. For $D = 13$, the verification route is to rebuild F, check its rivals, and check the refutation. For $D \geq 14$, it is to check the 34-case cover and inconsistencies, then the general embedding. The local lemma chain explains why these finite objects arise.

The accepted refutation is stored as seven ordered gzip files. Their concatenated decompressed stream is binary DRAT, with 2,049,035,259 bytes; the compressed total is 613,931,594 bytes. Verification uses the public `drat-trim` checker [2], with `-i -f`: explicit binary input from standard input and forward verification. The independent run builds with assertions enabled and returns `VERIFIED`, exit code zero, on the original F. Appendix A fixes the input, rivals and raw proof by SHA-256. The companion inventory specifies every part and its order.

7.1 Checking the LRAT with a verified program

The original F and all 34 auxiliary case CNFs also yield the exact success line `s VERIFIED UNSAT` from the official, unmodified ARMv8 build of `cake_lpr` [3], using LRAT proofs converted from the DRAT certificates. The $D = 13$ LRAT has 5,428,472,887 raw bytes and is stored in a

Obligation	Prose argument	Finite or certificate check
Attainment at 12	Unique maximal-helix decomposition	Two DP recurrences give optimum 12 and count 1; separate implementation agrees.
Normalization	Neighbor-set inclusion preserves the target and removes competitors	Alphabet relations and color symmetry checked on all letters.
Local replacements	Endpoint geometry, noncrossing and stack lower bound	Word predicates; all listed rivals tested from the definition.
All $D \geq 14$	Family-wise embedding, including shared separators	Rebuild auxiliary CNF; 68 word sets, 34 orbits, full-clause symmetries; 34 contradictions, with independent unit-propagation replay and additional LRAT checks by `cake_lpr`.
$D = 13$	A normalized design would satisfy F	Rebuild all 268,408 clauses, validate 227,280 rivals, check the DRAT refutation of F and its LRAT conversion with `cake_lpr`.

Table 2: Responsibility for the computer-assisted theorem. A checker acceptance alone does not supply the model-to-formula argument.

1,598,317,453-byte gzip file. A separate build replays the stored certificates without rerunning their converter and obtains the same success result. Corrupted-proof controls are rejected.

The HOL4 theorem `machine_code_sound` for the pinned CakeML revision connects the compiled checker to its parsed CNF semantics. In the usual two-argument CNF/proof mode, under its ARM execution, foreign-function-interface and filesystem assumptions, the success output implies that the parsed CNF is unsatisfiable. Successful process exit alone is insufficient. The DRAT-to-LRAT converter, including its local streaming changes, need not be trusted for this implication: `cake_lpr` checks the resulting proof against the input CNF.

The remaining trust includes HOL4's logic and kernel, the formal semantics, the correspondence between the ARM model and the machine, the C input/output interface, assembler/linker and operating system, and the provenance linking the distributed assembly to the proved compilation result. We used the distributed ARM assembly; we did not rebuild HOL4/CakeML or regenerate it from the proof. The normalization, rival legality, CNF/model correspondence, 34-case cover, general embedding and design DP remain the separate obligations in Table 2. The checker theorem does not itself formalize those RNA arguments.

8 Independent verification

A separate verification implementation used the statement, model and CNF specifications, the auxiliary embedding description, the rival lists and the certificate manifest. It did not use the producer's generation code, auxiliary proof-lifting explanation, prior check results or the coordinator's implementation. Public checker documentation and parser source were available to it. Audit details, dates and responsibility are recorded once in the companion `VERIFICATION-RECORD.md`.

This implementation recovered the 71-base design result, checked every rival, and regenerated both input CNFs byte for byte. It regenerated all seven auxiliary rival families, checked the entire word-set cover and the full-formula symmetries, and reproduced all 34 case contradictions. It checked the seven-part proof's hashes and complete raw stream, and accepted the $D = 13$ refutation. It also reviewed the embedding family by family. Tests of 50,199 mapped rivals supplement that general argument.

The three initial DRAT checks use the same public `drat-trim` implementation. The addi-

tional `cake_lpr` checks and replay supply a different checker with a proved correctness theorem, but are not part of the separate model-verification implementation described above: that implementation used `drat-trim` and did not rerun `cake_lpr`. The verification record distinguishes the roles. For the auxiliary cases, `drat-trim` accepts by unit propagation on the inputs, so this does not mean every supplied DRAT addition was exercised. Separate propagation traces and replay check that finite step as well. Normalization, the dynamic-programming decomposition and the embedding remain prose arguments. The full RNA theorem has not been formalized in a proof assistant, and this article has not undergone human peer review.

9 Scope

The maximum 12 concerns precisely (1), with every separator and hairpin length fixed as stated. It gives no maximum degree for arbitrary RNA trees, nor a classification of every $D \leq 12$. Changing the separator length can change the bound: the variant of (1) with all separators removed and one-base hairpins retained admits a design at $D = 25$.[1] This variant is being studied separately and is outside the claims and certificates of the present paper. If hairpins must contain three unpaired nucleotides, these targets themselves cease to be legal; lengthening them defines a different family. No conclusion about the Turner energy model, physical molecules or experimental folding follows from this theorem. The exact value is the proposed contribution within the reviewed literature; a priority claim is not established by a negative search.

Use of AI. Codex and Claude assisted the mathematical arguments, search, software, verification implementations, literature comparison and drafting. Tatsuhiko Takimoto is responsible for the manuscript. The computational and mathematical assurances are those stated above.

References

[1] Théo Boury, Laurent Bulteau and Yann Ponty. RNA Inverse Folding Under Stacked Base Pair Maximization. In *26th International Conference on Algorithms for Bioinformatics (WABI 2026)*, LIPIcs 390, 12:1–12:21, 2026. doi:10.4230/LIPIcs.WABI.2026.12.

[2] Marijn J. H. Heule and contributors. *The DRAT-trim proof checker*. Public software and documentation. `https://github.com/marijnheule/drat-trim`. The exact revision and build used for verification are recorded in the companion verification record.

[3] cake_lpr contributors and the CakeML project. *LPR proof checking with CakeML*. `https://github.com/tanyongkiam/cake_lpr`; *Verified proof checking*, `https://cakeml.org/checkers.html`. ARM correctness theorem: `lpr_arrayProofARM8.machine_code_sound`.

A Certificate identifiers

The following hashes identify the mathematical inputs and accepted proof, independently of local filenames. A rival-list hash refers to the uncompressed ASCII dot-bracket lines, including their newlines. The accompanying package specifies variable numbering, clause order and literal order, and rival-list order in `claims/SPEC.md` and `claims/RIVAL-UPPER.md`. The thirteen-word sets for all 34 cases, and the CNF and LRAT hashes, are in `data/U14-cases.json`. The names,

[1] The 124-nucleotide sequence `ACUCAGAUCAGACGCCGAGACUAGACUCCAG GCCAGGGGACCGGACCUUAAAACCGUACCGU UGCCAUGCCAGACUCGACUCAUCAUAUCAUU ACUAUACUACUCAGCUCAGAAAUUGCCGCGU` designs the target `((` followed by 24 copies of `((.))` and then `))`. This example can be checked with any maxStacks dynamic program implementing the model of Section 2. It is outside this article's claims and certificates.

order and individual hashes of the seven compressed $D = 13$ DRAT parts, and the large-proof inventory, are in `data/large-proofs.json`. The large proofs are distributed separately as release attachments. The article text alone is not sufficient for byte-exact regeneration; the serialization details are supplied in those package files.
$D = 13$ **input** F**:** 3,158 variables; 268,408 clauses; 20,535,942 bytes.
`4f8100bbe4362bf45db9b4be068eb115bf1c48c3f556ce5c3bceff1e7f5a4a6c`

$D = 13$ **rival list:** 227,280 structures.
`e1a149ce72fc055faa1e1f459ed1fd2f885df3d1c2795dab490154ac687f042a`

$D = 13$ **complete raw binary DRAT:** 2,049,035,259 bytes.
`ef0a5a3cd19984a3f8973740d247ea229c57044a39d8b4068895e3493b0aa116`

$\mathcal{U}_{14}$ **input:** 5,044 variables; 68,860 clauses.
`5ff81ea47c5fa1f66300587ea0d036cae32598c2070374e8ee423056f89f715f`

Auxiliary rival list: 1,731 structures.
`0ad72ae92ae2ab1f141d6a4ed213a0ac6df72cd154430fdccae30a64e370374c`

$D = 13$ **binary LRAT, gzip:** 1,598,317,453 bytes.
`8705abc77e86aa52856ef6c36f4bc01037693e7ef57fda96e0bf48f56cbc523a`

Expanded binary LRAT: 5,428,472,887 bytes.
`9b54d5df00f8db04a2d9d51688175ed75df683e54839c28db56fda714fddbd91`

Official `cake_lpr` revision:
`a36874a8b750b43fe4b385b8ddbf5b033e46a3fa`

ARMv8 checker executable SHA-256:
`2fee767af4478ed817f8bdf6feea5866e5e59f1f67c2a4477e98ee961612881d` The executable hash identifies this build, not every build on other systems. The companion record pins HOL4 and CakeML revisions and documents assembly and C-wrapper provenance.

For a local check, concatenate the decompressed proof parts in the inventory order and feed the resulting binary stream to `drat-trim F.cnf -i -f`. An input hash, a proof hash and a checker acceptance establish a propositional refutation of that input; the normalization, rival legality and model correspondence must also be checked as specified in Table 2.

To check the stored LRAT, decompress its gzip to the standard input of `cake_lpr`, with `F.cnf /dev/stdin` as the two checker arguments. The C wrapper used here accepts heap and stack options `-CML_HEAP_SIZE=4096` and `-CML_STACK_SIZE=128` before those arguments. Require the exact `s VERIFIED UNSAT` line and normal termination. Each auxiliary case must use its own CNF and corresponding LRAT, as identified in the inventory.